

Research topics

Hanyang University

Christian Rinderknecht

1st November 2012

Teaching

- ▶ All kinds of computer programming, in particular, functional languages (OCaml, Erlang, XSLT, Haskell etc.).
- ▶ Analysis and proofs of algorithms (formal methods).
- ▶ Compiler construction (automata theory, formal grammars etc.).
- ▶ Unix development tools: emacs, bash, make, perl, mercurial.
- ▶ Cutting edge: ePUB3
(HTML5+CSS+MathML+tablets/phones, under development).

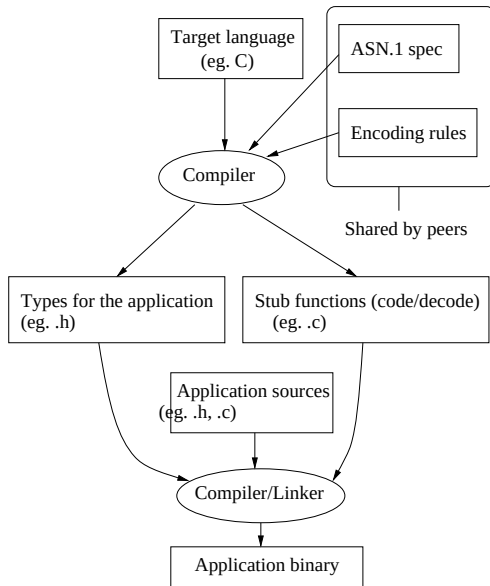
Research

Abstract Syntax Notation One (ASN.1) is a standardised language (ISO, ITU-T) for defining data types whose values may be exchanged across a network between two, possibly heterogeneous peers.

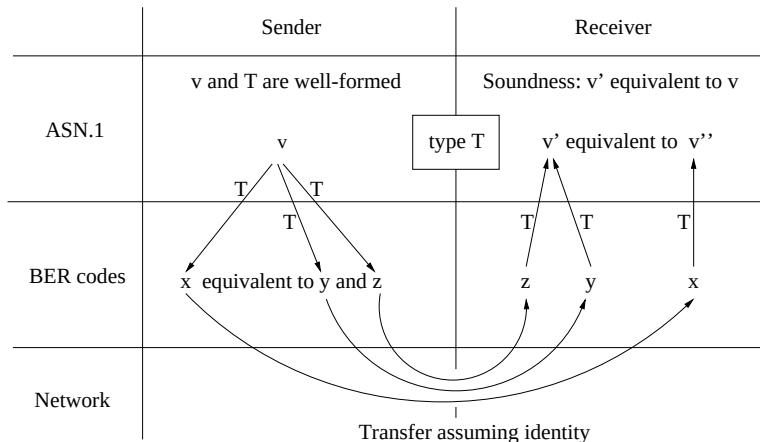
It is used in network management, secure email, mobile telephony, wireless applications, Long Term Evolution (4G LTE), electronic commerce, air traffic control, voice and video over the internet, digital certificates, radio paging, interactive television, financial service systems, biometrics, ATM transactions, call routing to local carriers (USA), plane take-offs and landings, Federal Express tracking network, Microsoft Internet Explorer, Outlook, databases (genetics, libraries), diagnostic monitoring systems (car factories), metro equipments etc.

Encoding rules specify how to serialize an ASN.1 value.

ASN.1 and the compilation chain



ASN.1/BER and telecommunication



Issues

1. Specifications: huge (704 pages A4 10pt), complex (concepts mutually dependent).
2. Syntax: large and ambiguous grammar, dynamic extensions.
3. English cannot tell the meaning of all combinations of syntactical constructs.
4. Types are value sets: subtypes yield general set constraints.
5. Encoding only applies to a subset of ASN.1.
6. Decoding is unspecified.
7. Dynamic checks in codecs (depending on the target programming language) are unspecified.
8. Open source or free compilers: extremely rare, unmaintained, fragile, without precise error reporting.

Issues in validation

ASN.1 types are considered as sets of values, thus types must have at least one (finite) value.

Because of the great expressivity of ASN.1, the compilers are not likely to fully check arbitrary combinations of subtyping constraints!

Vendors' argument: "This is hardly a real problem because those critical specifications are rarely found in practice."

My argument: "Maintenance costs are always wildly underestimated. Why not solve completely the problem, get a better product and save money and time?"

Problems

1. the types may have only infinite values:

```
T ::= SET {a T};
```

2. some value declarations may be ill-typed:

```
v REAL ::= "";
```

3. especially, some value references may be ill-typed, as

```
a VisibleString ::= b
```

```
b INTEGER ::= 0;
```

4. the subtype constraints may be inconsistent:

```
T ::= REAL (SIZE(7));
```

5. the subtypes may be empty, as

```
T ::= SET ((SIZE (1)) ^ (SIZE (2))) OF REAL;
```

6. the subtypes may have no value set:

```
T ::= REAL (ALL EXCEPT T).
```


Problems (cont.)

In other words, the issues to settle are respectively:

1. the finiteness problem;
2. the typechecking problem;
3. the type compatibility problem;
4. the constraint consistence problem;
5. the non-emptiness problem;
6. the solvability problem.

Analysis

- ▶ type compatibility $\subseteq$ typechecking;
- ▶ constraint consistence, non-emptiness $\subseteq$ solvability (the system is solved when we construct explicitly the values of each subtype);
- ▶ typechecking $\subseteq$ solvability:

$$y \text{ INTEGER } (0..9) ::= 1 \rightarrow \begin{cases} y \ A ::= 1 \\ A ::= \text{INTEGER } (0..9) \\ B ::= A \ (y) \end{cases}$$

where A and B are fresh type references.

So, finiteness and solvability are enough to get a full validation of ASN.1.

Two approaches

Every ASN.1 specification is rewritten in a subset of same expressiveness, with less syntactical constructs. Two strategies:

1. Few preliminary rewrites, followed by a collection of set constraints, then fed to a generic constraint solver.
2. Lots of preliminary rewrites, simple algorithms to check solvability. (This is like taking the previous method and interleave a dedicated solver with the syntactical rewrites.)

Comparison:

1. Pros: modularity, less programming. Cons: constraints too general for off-the-shelf solvers.
2. Pros: easy checking. Cons: heavily specialised, correctness hard to assess.

We need to find a trade-off.

Short presentation of ASN.1

Some basic types

- ▶ `ok BOOLEAN ::= TRUE`
- ▶ `zero INTEGER ::= 0`
`DayInTheYear ::= INTEGER {first(1), last(356)}`
`newYearsEve DayInTheYear ::= last`
- ▶ `SynchroIndicator ::= ENUMERATED {serial,parallel}`
`synchro SynchroIndicator ::= serial`
- ▶ `pi REAL ::= 3.14159`
`micron REAL ::= {mantissa 1,base 10,exponent -6}`
- ▶ `byte BIT STRING ::= '0D'H -- or '00001110'B`
`T ::= BIT STRING {msb(7), lsb(0)}`
`v T ::= {msb, lsb} -- or '10000001'B`
- ▶ For historical reasons, there are many string types.

Some constructed types

The SET type corresponds to the record-like structures in programming languages:

```
PersonInfo ::= SET {age INTEGER, married BOOLEAN}  
i PersonInfo ::= {married TRUE, age 42}
```

Some fields can be marked as optional or having a default value:

```
Point ::= SET {x REAL DEFAULT 0, y REAL DEFAULT 0}  
origin Point ::= {} -- or {x 0.0, y 0.0}
```

A real example:

```
DataAcknowledgementTPDU ::= SET {  
    destRef          Reference,  
    yr-tu-nr         TPDUnumber,  
    checkSum         CheckSum OPTIONAL,  
    subSeqNr         SubSequenceNumber DEFAULT 0,  
    flowControlCnf   FlowControlConfirmation OPTIONAL}
```

Some constructed types (cont.)

The SET OF type corresponds to the mathematical notion of sets with repetition:

```
T ::= SET OF INTEGER
```

```
empty T ::= {}
```

```
small T ::= {7, 9, 1, 1, 3}
```

Some constructed types (cont.)

The CHOICE type corresponds to a **union** in C.

```
T ::= CHOICE {x REAL, y BOOLEAN}
```

```
u T ::= x : 0.5
```

```
v T ::= y : FALSE
```

The Protocol Data Units (PDU) are CHOICE types, because they model all the possible queries and responses between two peers. A CHOICE type may be recursive, like the other constructed types. One real example (a Network Management Protocol) is:

```
CMISFilter ::= CHOICE {  
    item  FilterItem,  
    and   SET OF CMISFilter,  
    or    SET OF CMISFilter,  
    not   CMISFilter}
```

Some subtyping constraints

ASN.1 offers a very involved subtyping paradigm consisting of constraints upon recursive types, that restricts their corresponding sets of values in a set-theoretic manner, but also in a structural way.

- ▶ **Interval Constraint:** the INTEGER, REAL and (almost all) string types have totally ordered values, hence allowing interval definitions. For instance:

```
PositiveOrZeroInteger ::= INTEGER (0..MAX)
```

```
PositiveInteger ::= INTEGER (0<..MAX)
```

```
NegativeOrZeroInteger ::= INTEGER (MIN..0)
```

```
NegativeInteger ::= INTEGER (MIN.. $<0$ )
```

```
PositiveReal ::= REAL (0<..PLUS-INFINITY)
```

```
NegativeReal ::= REAL (MINUS-INFINITY.. $<0$ )
```

```
RealInterval ::= REAL (4e-5..1e-4)
```


Some subtyping constraints (cont.)

- ▶ **Value Constraint:** to restrict the set of values of a type to be a singleton:

```
Wednesday ::= Day (wednesday)
```

- ▶ **Union Constraint:** the new subtype contains the values of the first subtype and of the second subtype (keyword UNION or symbol |):

```
Day ::= ENUMERATED {monday, tuesday, wednesday,  
                    thursday, friday, saturday, sunday}
```

```
WeekEnd ::= Day (saturday | sunday)
```

- ▶ **Alphabet Constraint:** the strings can be restricted to be built upon a given alphabet:

```
CapitalAndSmall ::=
```

```
  IA5String (FROM ("A".."Z" | "a".."z"))
```

```
CapitalOrSmall ::=
```

```
  IA5String (FROM ("A".."Z") | FROM ("a".."z"))
```

Some subtyping constraints (cont.)

- **Size Constraint:** the values of string types may be constrained to a given sizes, introducing a constraint by the keyword `SIZE`:

```
Exactly31BitsString ::= BIT STRING (SIZE (31))
```

```
StringOf31BitsAtTheMost ::=  
    BIT STRING (SIZE (0..31))
```

```
NonEmptyString ::= BIT STRING (SIZE (1..MAX))
```

The size constraint can also apply to `SET OF` types. In that case, the semantics is very different: the values of the types are sets whose *cardinals* are specified by the size constraint:

```
SetOf5Strings ::=
```

```
    SET (SIZE(5)) OF PrintableString
```

```
SetOfStringsOf5Char ::=
```

```
    SET OF PrintableString (SIZE(5))
```

Some subtyping constraints (cont.)

- ▶ **Intersection Constraint:** the new subtype contains the values that belong to the two subtypes (keyword `INTERSECTION` or symbol `^`):

```
FrenchPhoneNumber ::=
  NumericString (FROM ("0".."9") ^ SIZE (10))
```

- ▶ **Inclusion Constraint:** to restrict a subtype to have only the values of a given subtype (optional keyword `INCLUDES`):

```
LongWeekEnd ::=
  Day ((INCLUDES (WeekEnd)) | monday)
Bis ::= Day (WeekEnd | monday)
```

Some subtyping constraints (cont.)

- ▶ **Complement Constraint:** to restrict the values of a type to *not* belong to another subtype:

```
Lipogramme ::=  
  IA5String (FROM (ALL EXCEPT ("e" | "E")))
```

- ▶ **Constraint on SET OF:** to restrict the elements of a SET OF value:

```
TextBlock ::= SET OF VisibleString  
AddressBlock ::=  
  TextBlock (WITH COMPONENT (SIZE(1..32)))
```

Some subtyping constraints (cont.)

- ▶ Partial Constraint: to restrict *some* fields of a SET or CHOICE:

```
Quadruple ::= SET {  
    alpha ENUMERATED {in, out} OPTIONAL,  
    beta  IA5String OPTIONAL,  
    gamma SET OF INTEGER,  
    delta BOOLEAN DEFAULT TRUE}
```

we can derive a subtype whose component alpha is always present and equals in, and the component gamma always has five elements:

```
Quadruple1 ::=  
    Quadruple (WITH COMPONENTS {...,  
                                                alpha (in) PRESENT,  
                                                gamma (SIZE (5)))})
```

Some subtyping constraints (cont.)

This subtype has the same values as:

```
Quadruple1 ::= SET {  
  alpha ENUMERATED {in, out} (in),  
  beta  IA5String OPTIONAL,  
  gamma SET SIZE (5) OF INTEGER,  
  delta BOOLEAN DEFAULT TRUE}
```